

P2093R5

Formatted output

Published Proposal, 2021-03-14

Author:

[Victor Zverovich](#)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

- 1 Introduction
- 2 Revision history
- 3 SG16 polls (R3)
- 4 LEWG polls (R1)
- 5 Motivating examples
- 6 API and naming
- 7 Unicode
- 8 Performance
- 9 Binary code
- 10 Impact on existing code
- 11 Implementation
- 12 Wording
- Appendix A: Unicode tests
- 13 Acknowledgements
- References
- Informative References

" $\|f\|_{\mathbb{C}} = \|\mathbb{E}\|_{\mathbb{C}} = \mathbb{E}$, $\|f\|_{\mathbb{C}} = \|\mathbb{E}\|_{\mathbb{C}} = \mathbb{E}$ "
— anonymous user

§ 1. Introduction

A new I/O-agnostic text formatting library was introduced in C++20 ([\[FORMAT\]](#)). This paper proposes integrating it with standard I/O facilities via a simple and intuitive API achieving the following goals:

- Usability

- Unicode support
- Good performance
- Small binary footprint

§ 2. Revision history

Changes since R4:

- Added SG16 Unicode poll results.
- Added a list of candidate headers formatted output functions can be added to.
- Moved the non-`ostream` overloads of formatted output functions to a separate header for cleaner separation of formatting and I/O facilities.

Changes since R3:

- Replaced `_isatty(_fileno(stream))` with `GetConsoleMode(_get_osfhandle(_fileno(stream)), ...)` in a note in [format.functions] because the former may return 1 for streams not referring to a terminal.

Changes since R2:

- Added better compatibility with other formatted I/O facilities as another advantage of using `stdout` in [§ 6 API and naming](#) per SG16 feedback.
- Clarified that [\[P1885\]](#) can be used for literal encoding detection per SG16 feedback.
- Added comparison of Unicode handling in various languages in [Appendix A: Unicode tests](#) and a summary in [§ 7 Unicode](#) per SG16 request.
- Removed incorrect "exposition-only" in [§ 7 Unicode](#).
- Replaced "both source and literal encodings are UTF-8, which is enabled by the `/utf-8` compiler flag" with "the literal (execution) encoding is UTF-8, which is enabled by the `/execution-charset:utf-8` compiler flag" since the source encoding is irrelevant there.
- Rephrased *Effects* of `vprint_unicode` for clarity.

Changes since R1:

- Added missing `println` overloads that take `FILE*` and `ostream&`.
- Moved the print functions that take `ostream&` to `<ostream>` since `<format>` shouldn't depend on `<ostream>`.
- Clarified why it is useful to provide `vprint*` functions in [§ 9 Binary code](#).
- Rebased the wording onto the latest working draft, N4861, in particular updating the *Throws* clauses to match existing wording.

- Replaced `std::system_error` with `system_error` in the wording.
- Added paragraph numbers to the wording.

Changes since R0:

- Clarified that adding `wchar_t` overloads will be addressed in a separate paper per the UK C++ panel feedback.

§ 3. SG16 polls (R3)

Forward P2093R3 to LEWG.

SF	WF	N	WA	SA
4	2	2	0	1

Consensus? Yes

§ 4. LEWG polls (R1)

We prefer `std::cout` as the default output target.

SF	WF	N	WA	SA
3	2	6	6	5

Add a member function on `ostream` instead of a `std::print(ostream&, ...)` free function overload.

SF	WF	N	WA	SA
0	2	5	17	3

Consensus against.

Remove `std::println` from the paper?

SF	WF	N	WA	SA
1	10	7	4	4

No consensus for change.

We are happy with the design with regards to UTF-8 output.

Unanimous consent.

§ 5. Motivating examples

Consider a common task of printing formatted text to `stdout`:

C++20	Proposed
<pre>std::cout << std::format("Hello, {}!", name);</pre>	<pre>std::print("Hello, {}!", name);</pre>

The proposed `std::print` function improves usability, avoids allocating a temporary `std::string` object and calling `operator<<` which performs formatted I/O on text that is already formatted. The number of function calls is reduced to one which, together with `std::vformat`-like type erasure, results in much smaller binary code (see § 9 Binary code).

Existing alternatives in C++20:

Code	Comments
<pre>std::cout << "Hello, " << name << "!";</pre>	Requires even more formatted I/O function calls; message is interleaved with parameters; can result in interleaved output.
<pre>std::printf("Hello, %s!", name);</pre>	Only works if <code>name</code> is a null-terminated character string.
<pre>auto msg = std::format("Hello, {}!", name); std::fputs(msg.c_str(), stdout);</pre>	Constructs a temporary string; requires a call to <code>c_str()</code> and a separate I/O function call, although potentially cheaper than <code>operator<<</code> .

Another problem is formatting of Unicode text:

```
std::cout << "Привет, κόσμος!";
```

If the source and execution encoding is UTF-8 this will produce the expected output on most GNU/Linux and macOS systems. Unfortunately on Windows it is almost guaranteed to produce [mojibake](#) despite the fact that the system is fully capable of printing Unicode, for example

$$\|f\|_{\mathcal{C}} = \|g\|_{\mathcal{C}} = \epsilon, \quad \|f\|_{\mathcal{C}} = \hat{a} \|g\|_{\mathcal{C}} = \epsilon!$$

even when compiled with `/utf-8` using Visual C++ ([[MSVC-UTF8](#)]). This happens because the terminal assumes code page 437 in this case independently of the execution encoding.

With the proposed paper

```
std::print("Привет, κόσμος!");
```

will print "Привет, κόσμος!" as expected allowing programmers to write Unicode text portably using standard facilities. This will bring C++ on par with other languages where such functionality has been available for a long time. For comparison this just works in Python 3.8 on Windows with the same active code page and console settings:

```
>>> print("Привет, κόσμος!")
Привет, κόσμος!
```

This problem is independent of formatting `char8_t` strings but the same solution applies there. Adding `charN_t` and `wchar_t` overloads will be explored in a separate paper in a more general context.

§ 6. API and naming

Many programming languages provide functions for printing text to standard output, often combined with formatting:

Language	Function(s)
C	<code>printf</code> [N2176]
C#/NET	<code>Console.Write</code> [DOTNET-WRITE]
COBOL	<code>DISPLAY</code> statement [N0147]
Fortran	<code>print</code> and <code>write</code> statements [N2162]
Go	<code>Printf</code> [GO-FMT]
Java	<code>PrintStream.format</code> , <code>PrintStream.print</code> , <code>PrintStream.printf</code> [JAVA-PRINT]
JavaScript	<code>console.log</code> [WHATWG-CONSOLE]
Perl	<code>printf</code> [PERL-PRINTF]
PHP	<code>printf</code> [PHP-PRINTF]
Python	<code>print</code> statement or function [PY-FUNC]
R	<code>print</code> [R-PRINT]
Ruby	<code>print</code> and <code>printf</code> [RUBY-PRINT]
Rust	<code>print!</code> [RUST-PRINT]
Swift	<code>print</code> [SWIFT-PRINT]

Variations of `print[f]` appear to be the most popular naming choice for this functionality. It is either provided as a free function (most common) or a member function (less common) together with a global object representing standard output stream. Notable exceptions are COBOL, Fortran, and Python 2 which have dedicated language statements and Rust where `print!` is a function-like macro.

We propose adding a free function called `print` with overloads for writing to the standard output (the default) and an explicitly passed output stream object. The default output stream can be either `stdout` or `std::cout`. We propose using `stdout` for the following reasons:

- `stdout` is considerably faster on at least two major implementations (see [§ 8 Performance](#)).
- Better compatibility with other formatted I/O facilities compared to `std::cout` and its associated `std::streambuf` that suffer from private buffering, localization and conversion services that must be synchronized at a lower level.
- `print` won't use any formatted output functionality of `ostream`.

Since `stdout` doesn't have an associated locale we propose using the current global locale for locale-specific formatting which is consistent with `format`. With `cout` or another explicitly passed stream, the stream's locale will be used. In all cases the default formatting is locale-independent.

Another option is to make `print` a member function of `basic_ostream`. This would make usage somewhat more awkward:

```
std::cout.print("Hello, {}!", name);
```

A free function can also be overloaded to take `FILE*` to simplify migration (possibly automated) of code from `printf` to the new facility.

There are multiple approaches to appending a trailing newline:

- Don't append a newline automatically: `printf` in C and other languages.
- Append a newline but don't format arguments: `puts` in C (inconsistent with `fputs`).
- Have two formatting functions/macros, one that appends newline and another that doesn't: `print/println` in Java, `print!/println!` in Rust, `Printf/Println` in Go, `Write/WriteLine` in C#/.NET.
- Let the user choose a terminating string defaulting to `"\n"` and do limited formatting: `print` in Python and Swift.

We propose not appending a newline automatically for consistency with `printf` and iostreams:

```
std::print("Hello, {}!", name);    // doesn't print a newline
std::print("Hello, {}!\n", name);  // prints a newline
```

Additionally we can provide a function that appends a newline:

```
std::println("Hello, {}!", name);  // prints a newline
```

Although `println` doesn't provide much usability improvement compared to `print` with explicit `'\n'`, it has been a frequently requested feature in the `fmt` library ([\[FMT\]](#)).

Another question is which header non-`ostream` overloads of formatted output functions should go to. Possible options:

- `<io>`

- `<print>`
- `<format>`
- `<ostream>`
- `<utility>`

The current paper proposes `<io>` analogous to `<cstdio>` so that the future I/O facilities that don't depend on `ostream` could be added there. Using `<ostream>` is undesirable because this header and its transitive dependencies are very big (42 thousand lines preprocessed on `libc++`):

```
% echo '#include <ostream>' | clang++ -E -x c++ - | wc -l
42491
```

It also pulls in a lot of unrelated symbols such as `ostream` insertion operators, global `cout`, `cerr`, `clog` variables and their `wchar_t` counterparts.

`ostream` overloads are added to the `<ostream>` header.

§ 7. Unicode

We can prevent mojibake in the Unicode example by detecting if the string literal encoding is UTF-8 and dispatching to a different function that correctly handles Unicode, for example:

```
constexpr bool is_utf8() {
    const unsigned char micro[] = "\u00B5";
    return sizeof(micro) == 3 && micro[0] == 0xC2 && micro[1] == 0xB5;
}

template <typename... Args>
void print(string_view fmt, const Args&... args) {
    if (is_utf8())
        vprint_unicode(fmt, make_format_args(args...));
    else
        vprint_nonunicode(fmt, make_format_args(args...));
}
```

where the `vprint_unicode` function formats and prints text in the UTF-8 encoding using the native system API that supports Unicode and `vprint_nonunicode` does the same for other encodings. The latter ensures that interoperability with code using legacy encodings is preserved even though `print` is a new API and it is not strictly necessary.

In Visual C++ `is_utf8` will return `true` if the literal (execution) encoding is UTF-8, which is enabled by the `/execution-charset:utf-8` compiler flags or other means, and `false` otherwise. Literal encoding detection can be implemented in a more elegant way using [\[P1885\]](#).

This approach has been implemented in the `fmt` library ([\[FMT\]](#)) and successfully tested on a variety of platforms.

Here's an example output on Windows:

```
C:\projects\fmt>cl print-test.cc /I include /utf-8 /EHsc
Microsoft (R) C/C++ Optimizing Compiler Version 19.24.28316 for x86
Copyright (C) Microsoft Corporation. All rights reserved.

print-test.cc
Microsoft (R) Incremental Linker Version 14.24.28316.0
Copyright (C) Microsoft Corporation. All rights reserved.

/out:print-test.exe
print-test.obj

C:\projects\fmt>print-test
Привет, κόσμος!
```

At the same time interoperability with legacy code is preserved when literal encoding is not UTF-8. In particular, in case of EBCDIC, Shift JIS or a non-Unicode Windows code page, `print` will perform no transcoding and the text will be printed as is.

The following table summarizes the behavior of formatted output facilities in different programming languages:

Language	Linux		macOS		Windows	
	Terminal	Redirect	Terminal	Redirect	Terminal	Redirect
C	Correct	UTF-8	Correct	UTF-8	Wrong	UTF-8
Go	Correct	UTF-8	Correct	UTF-8	Correct	UTF-8
Java	Correct	UTF-8*	Correct	UTF-8*	Wrong	CP1251 (lossy)
JavaScript	Correct	UTF-8*	Correct	UTF-8*	Correct	UTF-8*
Python	Correct	UTF-8*	Correct	UTF-8*	Correct	Error
Rust	Correct	UTF-8	Correct	UTF-8	Correct	UTF-8

* - the output is transcoded from a different UTF representation.

Correct means that the test message "Привет, κόσμος!" was fully readable in the terminal output. None of the tested language facilities were able to produce readable output when piped through the standard `findstr` command on Windows. Java gave the worst results producing both mojibake and replacement characters in this case: "ᐅᐅᐅᐅᐅᐅᐅ, ??????". Most other languages produced valid UTF-8 when the output of `findstr` was redirected to a file.

The current paper proposes following C, Go, JavaScript and Rust and preserve the original encoding (modulo UTF conversion). The only difference compared to `printf` is that we fix the output to console on Windows. Java's approach is problematic for the following reasons:

- There is a silent data loss for **valid** Unicode code points when the output is redirected to a file.

- It is more expensive because of transcoding.
- It may give an unusable result when piped through standard Windows commands like `findstr`.
- It transcodes into legacy encodings that are rarely used in practice nowadays. For example, usage of CP1251 dropped from 4.3% to 0.9% in the last 10 years ([\[ENCODING-TRENDS\]](#)).

The full listings of test programs are given in [Appendix A: Unicode tests](#).

§ 8. Performance

All the performance benefits of `std::format` ([\[FORMAT\]](#)) automatically carry over to this proposal. In particular, locale-independence by default reduces global state and makes formatting more efficient compared to `stdio` and `iostreams`. There are fewer function calls (see [§ 9 Binary code](#)) and no shared formatting state compared to `iostreams`.

The following benchmark compares the reference implementation of `print` with `printf` and `ostream`. This benchmark formats a simple message and prints it to the output stream redirected to `/dev/null`. It uses the Google Benchmark library [\[GOOGLE-BENCH\]](#) to measure timings:

```

#include <cstdio>
#include <iostream>

#include <benchmark/benchmark.h>
#include <fmt/ostream.h>

void printf(benchmark::State& s) {
    while (s.KeepRunning())
        std::printf("The answer is %d.\n", 42);
}
BENCHMARK(printf);

void ostream(benchmark::State& s) {
    std::ios::sync_with_stdio(false);
    while (s.KeepRunning())
        std::cout << "The answer is " << 42 << ".\n";
}
BENCHMARK(ostream);

void print(benchmark::State& s) {
    while (s.KeepRunning())
        fmt::print("The answer is {}.\n", 42);
}
BENCHMARK(print);

void print_cout(benchmark::State& s) {
    std::ios::sync_with_stdio(false);
    while (s.KeepRunning())
        fmt::print(std::cout, "The answer is {}.\n", 42);
}
BENCHMARK(print_cout);

void print_cout_sync(benchmark::State& s) {
    std::ios::sync_with_stdio(true);
    while (s.KeepRunning())
        fmt::print(std::cout, "The answer is {}.\n", 42);
}
BENCHMARK(print_cout_sync);

BENCHMARK_MAIN();

```

The benchmark was compiled with Apple clang version 11.0.0 (clang-1100.0.33.17) with `-O3 -DNDEBUG` and run on macOS 10.15.4. Below are the results:

Run on (8 X 2800 MHz CPU s)

CPU Caches:

L1 Data 32K (x4)

L1 Instruction 32K (x4)

L2 Unified 262K (x4)

L3 Unified 8388K (x1)

Load Average: 1.83, 1.88, 1.82

Benchmark	Time	CPU	Iterations
printf	87.0 ns	86.9 ns	7834009
ostream	255 ns	255 ns	2746434
print	78.4 ns	78.3 ns	9095989
print_cout	89.4 ns	89.4 ns	7702973
print_cout_sync	91.5 ns	91.4 ns	7903889

Both `print` and `printf` are ~3 times faster than `cout` even with synchronization to the standard C streams turned off. `print` is 14% faster when printing to `stdout` than to `cout`. For this reason and because `print` doesn't use formatting facilities of `ostream` we propose using `stdout` as the default output stream and providing an overload for writing to `ostream`.

On Windows 10 with Visual C++ 2019 the results are similar although the difference between `print` writing to `stdout` and `cout` is smaller with `stdout` being 7% faster:

Run on (1 X 2808 MHz CPU)

CPU Caches:

L1 Data 32K (x1)

L1 Instruction 32K (x1)

L2 Unified 262K (x1)

L3 Unified 8388K (x1)

Benchmark	Time	CPU	Iterations
printf	835 ns	816 ns	746667
ostream	2410 ns	2400 ns	280000
print	580 ns	572 ns	1120000
print_cout	623 ns	614 ns	1120000
print_cout_sync	615 ns	614 ns	1120000

§ 9. Binary code

We propose minimizing per-call binary code size by applying the type erasure mechanism from [P0645]. In this approach all the formatting and printing logic is implemented in a non-variadic function `vprint`. Inline variadic

`print` function only constructs a `format_args` object, representing an array of type-erased argument references, and passes it to `vprint*`. Here is a simplified example:

```
void vprint(string_view fmt, format_args args);

template<class... Args>
inline void print(string_view fmt, const Args&... args) {
    return vprint(fmt, make_format_args(args...));
}
```

We provide `vprint*` overloads so that users can apply the same technique to their own code. For example:

```
void vlog(log_level level, string_view fmt, format_args args) {
    // Print the log level and use vprint* overloads to format and print the
    // message.
}

template<class... Args>
inline void log(log_level level, string_view fmt, const Args&... args) {
    return vlog(level, fmt, make_format_args(args...));
}
```

Here `vlog` that implements the logging logic is not parameterized on formatting argument types resulting in less code bloat compared to a naive templated version. As a real-world example, this technique has been applied in the Folly Logger ([\[FOLLY\]](#)) bringing ~5x binary size reduction per logging function call.

Below we compare the reference implementation of `print` to standard formatting facilities. All the code snippets are compiled with clang (Apple clang version 11.0.0 clang-1100.0.33.17) with `-O3 -DNDEBUG -c -std=c++17` and the resulting binaries are disassembled with `objdump -S`:

```
void printf_test(const char* name) {
    printf("Hello, %s!", name);
}
```

```
__Z11printf_testPKc:
0:      55      pushq   %rbp
1:      48 89 e5      movq    %rsp, %rbp
4:      48 89 fe      movq    %rdi, %rsi
7:      48 8d 3d 08 00 00 00      leaq    8(%rip), %rdi
e:      31 c0      xorl    %eax, %eax
10:     5d      popq    %rbp
11:     e9 00 00 00 00      jmp     0 <__Z11printf_testPKc+0x16>
```

```

void ostream_test(const char* name) {
    std::cout << "Hello, " << name << "!";
}

```

__Z12ostream_testPKc:

```

0:      55      pushq   %rbp
1:      48 89 e5      movq    %rsp, %rbp
4:      41 56      pushq   %r14
6:      53      pushq   %rbx
7:      48 89 fb      movq    %rdi, %rbx
a:      48 8b 3d 00 00 00 00      movq    (%rip), %rdi
11:     48 8d 35 6c 03 00 00      leaq    876(%rip), %rsi
18:     ba 07 00 00 00      movl    $7, %edx
1d:     e8 00 00 00 00      callq   0 <__Z12ostream_testPKc+0x22>
22:     49 89 c6      movq    %rax, %r14
25:     48 89 df      movq    %rbx, %rdi
28:     e8 00 00 00 00      callq   0 <__Z12ostream_testPKc+0x2d>
2d:     4c 89 f7      movq    %r14, %rdi
30:     48 89 de      movq    %rbx, %rsi
33:     48 89 c2      movq    %rax, %rdx
36:     e8 00 00 00 00      callq   0 <__Z12ostream_testPKc+0x3b>
3b:     48 8d 35 4a 03 00 00      leaq    842(%rip), %rsi
42:     ba 01 00 00 00      movl    $1, %edx
47:     48 89 c7      movq    %rax, %rdi
4a:     5b      popq    %rbx
4b:     41 5e      popq    %r14
4d:     5d      popq    %rbp
4e:     e9 00 00 00 00      jmp     0 <__Z12ostream_testPKc+0x53>
53:     66 2e 0f 1f 84 00 00 00 00 00      nopw    %cs:(%rax,%rax)
5d:     0f 1f 00      nopl    (%rax)

```

```

void print_test(const char* name) {
    print("Hello, {}!", name);
}

```

```

__Z10print_testPKc:
    0:      55      pushq   %rbp
    1:      48 89 e5      movq   %rsp, %rbp
    4:      48 83 ec 10    subq   $16, %rsp
    8:      48 89 7d f0    movq   %rdi, -16(%rbp)
    c:      48 8d 3d 19 00 00 00    leaq   25(%rip), %rdi
    13:     48 8d 4d f0    leaq   -16(%rbp), %rcx
    17:     be 0a 00 00 00    movl   $10, %esi
    1c:     ba 0d 00 00 00    movl   $13, %edx
    21:     e8 00 00 00 00    callq  0 <__Z10print_testPKc+0x26>
    26:     48 83 c4 10    addq   $16, %rsp
    2a:     5d      popq    %rbp
    2b:     c3      retq

```

The code generated for the `print_test` function that uses the reference implementation of `print` described in this proposal is more than 2x smaller than the `ostream` code and has one function call instead of three. The `printf` code is further 2x smaller but doesn't have any error handling. Adding error handling would make its code size closer to that of `print`.

The following factors contribute to the difference in binary code size between `print` and `printf`:

- Passing format string as `string_view` instead of `const char*`.
- Capturing and passing argument type information.
- Preparing the array of formatting arguments.

§ 10. Impact on existing code

The current proposal adds new functions to the headers `<io>` and `<ostream>` and should have no impact on existing code.

§ 11. Implementation

The proposed `print` function has been implemented in the the open-source fmt library [\[FMT\]](#) and has been in use for about 6 years.

§ 12. Wording

Add an entry for `__cpp_lib_print` to section "Header `<version>` synopsis [\[version.syn\]](#)", in a place that respects the table's current alphabetic order:

```
#define __cpp_lib_print 202005L **placeholder** // also in <format>
```

Add the header `<io>` to the "C++ library headers" table in [headers], in a place that respects the table's current alphabetic order.

29.7.? Header `<io>` synopsis [io.syn].

```
namespace std {
    template<class... Args>
        void print(string_view fmt, const Args&... args);
    template<class... Args>
        void print(FILE* stream, string_view fmt, const Args&... args);

    template<class... Args>
        void println(string_view fmt, const Args&... args);
    template<class... Args>
        void println(FILE* stream, string_view fmt, const Args&... args);

    void vprint_unicode(string_view fmt, format_args args);
    void vprint_unicode(FILE* stream, string_view fmt, format_args args);

    void vprint_nonunicode(string_view fmt, format_args args);
    void vprint_nonunicode(FILE* stream, string_view fmt, format_args args);
}
```

Modify section "Header `<ostream>` synopsis [ostream.syn]":

```
...
template<class charT, class traits, class T>
basic_ostream<charT, traits>& operator<< (basic_ostream<charT, traits>&& os, const T& t) {
    return os<<t;
}

template<class... Args>
    void print(ostream& os, string_view fmt, const Args&... args);
template<class... Args>
    void println(ostream& os, string_view fmt, const Args&... args);

void vprint_unicode(ostream& os, string_view fmt, format_args args);
void vprint_nonunicode(ostream& os, string_view fmt, format_args args);
```

29.7.? Print functions [print.fun].

```
template<class... Args>
    void print(string_view fmt, const Args&... args);
```


Effects: Equivalent to:

```
print(stdout, fmt, make_format_args(args...));
```

```
template<class... Args>  
void print(FILE* stream, string_view fmt, const Args&... args);
```

27 *Effects:* If string literal encoding is UTF-8, equivalent to:

```
vprint_unicode(stream, fmt, make_format_args(args...));
```

Otherwise, equivalent to:

```
vprint_nonunicode(stream, fmt, make_format_args(args...));
```

```
template<class... Args>  
void println(string_view fmt, const Args&... args);
```

28 *Effects:* Equivalent to:

```
print("{}\n", format(fmt, args...));
```

```
template<class... Args>  
void println(FILE* stream, string_view fmt, const Args&... args);
```

29 *Effects:* Equivalent to:

```
print(stream, "{}\n", format(fmt, args...));
```

```
void vprint_unicode(string_view fmt, format_args args);
```

30 *Effects:* Equivalent to:

```
vprint_unicode(stdout, fmt, args);
```

```
void vprint_unicode(FILE* stream, string_view fmt, format_args args);
```

31 *Effects:* Let `out = vformat(fmt, args)`. If `stream` refers to a terminal [Note: On POSIX and Windows meaning that `isatty(fileno(stream))` and `GetConsoleMode(_get_osfhandle(_fileno(stream)), ...)` return nonzero respectively. — end note] capable of displaying Unicode, writes `out` to the terminal using

the native Unicode API. [Note: On Windows this API is `WriteConsoleW`. — end note] If this requires transcoding then invalid code points are substituted with U+FFFD  REPLACEMENT CHARACTER. Otherwise writes `out` to `stream` unchanged.

Throws: As specified in `[format.err.report]` or `system_error` if a call by the implementation to an operating system or other underlying API results in an error that prevents the function from meeting its specifications.

```
void vprint_nonunicode(string_view fmt, format_args args);
```

32 *Effects:* Equivalent to:

```
vprint_nonunicode(stdout, fmt, args);
```

```
void vprint_nonunicode(FILE* stream, string_view fmt, format_args args);
```

33 *Effects:* Writes the result of `vformat(fmt, args)` to `stream`.

Throws: As specified in `[format.err.report]` or `system_error` if a call by the implementation to an operating system or other underlying API results in an error that prevents the function from meeting its specifications.

Add subsection "Print `[ostream.formatted.print]`" to "Formatted output functions `[ostream.formatted]`":

```
template<class... Args>
void print(ostream& os, string_view fmt, const Args&... args);
```

1 *Effects:* If string literal encoding is UTF-8, equivalent to:

```
vprint_unicode(os, fmt, make_format_args(args...));
```

Otherwise, equivalent to:

```
vprint_nonunicode(os, fmt, make_format_args(args...));
```

```
void vprint_unicode(ostream& os, string_view fmt, format_args args);
```

2 *Effects:* Let `out = vformat(os.getloc(), fmt, args)`. If `os` is a file stream (its associated stream buffer is an instance of `basic_filebuf`) that refers to a terminal capable of displaying Unicode, writes `out` transcoded to the native system Unicode encoding to the terminal using the native API that preserves the encoding. Otherwise writes `out` to `stream` without transcoding.

Throws: As specified in `[format.err.report]` or `system_error` if a call by the implementation to an operating system or other underlying API results in an error that prevents the function from meeting its specifications.

```
void vprint_nonunicode(ostream& os, string_view fmt, format_args args);
```

3 *Effects:* Writes the result of `vformat(os.getloc(), fmt, args)` to `os`.

Throws: As specified in [\[format.err.report\]](#) or `system_error` if a call by the implementation to an operating system or other underlying API results in an error that prevents the function from meeting its specifications.

§ Appendix A: Unicode tests

This appendix gives full listings of programs for testing Unicode handling in various formatting facilities as well as test commands and their output on different platforms. The code contains additional sanity checks to ensure that the strings are encoded in some form of UTF as opposed to a legacy encoding.

C (`test.c`):

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    const char* message = "Привет, κόσμος!\n";
    if ((unsigned char)message[0] != 0xD0 && (unsigned char)message[1] != 0x9F)
        abort();
    printf(message);
}
```

Go (`test.go`):

```
package main

import "fmt"
import "log"

func main() {
    var message = "Привет, κόσμος!"
    if message[0] != 0xD0 && message[1] != 0x9F {
        log.Fatal("wrong encoding")
    }
    fmt.Println(message)
}
```

Java (`Test.java`):

```

class Test {
    public static void main(String[] args) {
        String message = "Привет, κόσμος!\n";
        if (message.charAt(0) != 0x41F) throw new RuntimeException();
        System.out.print(message);
    }
}

```

JavaScript / Node.js (`test.js`):

```

message = "Привет, κόσμος!";
if (message.charCodeAt(0) != 0x41F) throw "wrong encoding";
console.log(message);

```

Python (`test.py`):

```

message = "Привет, κόσμος!"
if ord(message[0]) != 0x41F:
    raise Exception()
print(message)

```

Rust (`test.rs`):

```

fn main() {
    if "Привет, κόσμος!".chars().nth(0).unwrap() as u32 != 0x41F {
        panic!();
    }
    println!("Привет, κόσμος!");
}

```

Linux:

```
$ cc test.c -o c-test
$ ./c-test
Привет, κόσμος!
$ ./c-test > out-c-linux.txt

$ go build -o go-test test.go
$ ./go-test
Привет, κόσμος!
$ ./go-test > out-go-linux.txt

$ java Test
Привет, κόσμος!
$ java Test > out-java-linux.txt

$ node test.js
Привет, κόσμος!
$ node test.js > out-js-linux.txt

$ python3 test.py
Привет, κόσμος!
$ python3 test.py > out-py-linux.txt

$ rustc test.rs -o rust-test
$ ./rust-test
Привет, κόσμος!
$ ./rust-test > out-rust-linux.txt
```

All output files are in UTF-8:

- [out-c-linux.txt](#)
- [out-go-linux.txt](#)
- [out-java-linux.txt](#)
- [out-js-linux.txt](#)
- [out-py-linux.txt](#)
- [out-rust-linux.txt](#)

Linux configuration:

- Ubuntu Focal 20.04 with the ru_RU.UTF-8 locale
- cc: gcc 9.3.0
- go: go1.13.8
- java: openjdk 11.0.9.1
- node: v14.5.0

- python3: 3.7.5
- rustc: 1.47.0

macOS:

```
% cc test.c -o c-test
% ./c-test
Привет, κόσμος!
% ./c-test > out-c-macos.txt

% go build -o test-go test.go
% ./test-go
Привет, κόσμος!
% ./test-go > out-go-macos.txt

% java Test
Привет, κόσμος!
% java Test > out-java-macos.txt

% node test.js
Привет, κόσμος!
% node test.js > out-js-macos.txt

% python3 test.py
Привет, κόσμος!
% python3 test.py > out-py-macos.txt

% rustc test.rs -o rust-test
% ./rust-test
Привет, κόσμος!
% ./rust-test > out-rust-macos.txt
```

All output files are in UTF-8:

- [out-c-macos.txt](#)
- [out-go-macos.txt](#)
- [out-java-macos.txt](#)
- [out-js-macos.txt](#)
- [out-py-macos.txt](#)
- [out-rust-macos.txt](#)

macOS configuration:

- macOS Catalina 10.15.7 with the ru_RU.UTF-8 locale which is the default for Russian

- cc: Apple clang version 12.0.0 (clang-1200.0.32.27)
- go: go1.15.5
- java: openjdk 14.0.1
- node: v14.5.0
- python3: 3.7.5
- rustc: 1.47.0

Windows:

```

>cl /Fe:c-test.exe test.c
...
>c-test
Привет, κόσμος!
>c-test > out-c-windows.txt
>c-test | findstr ,
Привет, κόσμος!

>go build -o go-test.exe test.go
>go-test
Привет, κόσμος!
>go-test > out-go-windows.txt
>go-test | findstr ,
Привет, κόσμος!

>java Test
Привет, ??????
>java Test > out-java-windows.txt
>java Test | findstr ,
Привет, ??????

>node test.js
Привет, κόσμος!
>node test.js > out-js-windows.txt
>node test.js | findstr ,
Привет, κόσμος!

>python test.py
Привет, κόσμος!
>python test.py > out-py-windows.txt
Traceback (most recent call last):
  File "...\\test.py", line 4, in <module>
    print(message)
  File "...\\Python39\\lib\\encodings\\cp1251.py", line 19, in encode
    return codecs.charmap_encode(input,self.errors,encoding_table)[0]
UnicodeEncodeError: 'charmap' codec can't encode characters in position 8-13: chara
>python test.py | findstr ,
Traceback (most recent call last):
  File "...\\test.py", line 4, in <module>
    print(message)
  File "...\\Python39\\lib\\encodings\\cp1251.py", line 19, in encode
    return codecs.charmap_encode(input,self.errors,encoding_table)[0]
UnicodeEncodeError: 'charmap' codec can't encode characters in position 8-13: chara

>rustc test.rs -o rust-test.exe
>rust-test
Привет, κόσμος!

```



```
>rust-test > out-rust-windows.txt
>rust-test | findstr ,
  11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884 885 886 887 888 889 890 891 892 893 894 895 896 897 898 899 900 901 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918 919 920 921 922 923 924 925 926 927 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 944 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 968 969 970 971 972 973 974 975 976 977 978 979 980 981 982 983 984 985 986 987 988 989 990 991 992 993 994 995 996 997 998 999 1000 1001 1002 1003 1004 1005 1006 1007 1008 1009 1010 1011 1012 1013 1014 1015 1016 1017 1018 1019 1020 1021 1022 1023 1024 1025 1026 1027 1028 1029 1030 1031 1032 1033 1034 1035 1036 1037 1038 1039 104
```

C, JavaScript (node.js), Rust and Go produced valid UTF-8 when the output was redirected to files. Java produced a file in the legacy CP1251 encoding with for non-representable code points. Python failed on transcoding to CP1251. Output files:

- [out-c-windows.txt](#)
- [out-go-windows.txt](#)
- [out-java-windows.txt](#)
- [out-js-windows.txt](#)
- [out-py-windows.txt](#)
- [out-rust-windows.txt](#)

Windows configuration:

- Windows 10 10.0.19041 with Russian Region and Language settings
- cl: Microsoft (R) C/C++ Optimizing Compiler Version 19.28.29335
- go: go1.15.6
- java: Java HotSpot(TM) 64-Bit Server VM (build 15.0.1+9-18)
- node: v14.15.3
- python: 3.9.1
- rustc: v14.15.3

§ 13. Acknowledgements

Thanks to Corentin Jabot for his work on text encodings in C++ and in particular [\[P1885\]](#) that will simplify implementation of the current proposal.

Thanks to Roger Orr, Peter Brett, the BSI C++ panel and Tom Honermann for their feedback, support, constructive criticism and contributions to the proposal.

§ References

§ Informative References

[DOTNET-WRITE]

.NET documentation, [Console.Write Method](https://docs.microsoft.com/en-us/dotnet/api/system.console.write). URL: <https://docs.microsoft.com/en-us/dotnet/api/system.console.write>

[ENCODING-TRENDS]

Historical yearly trends in the usage statistics of character encodings for websites. URL: https://w3techs.com/technologies/history_overview/character_encoding/ms/y

[FMT]

Victor Zverovich; et al. [The fmt library](https://github.com/fmtlib/fmt). URL: <https://github.com/fmtlib/fmt>

[FOLLY]

Folly: Facebook Open-source Library. URL: <https://github.com/facebook/folly>

[FORMAT]

Richard Smith. [Working Draft, Standard for Programming Language C++, Formatting \[format\]](https://wg21.link/n4861#section.20.20). URL: <https://wg21.link/n4861#section.20.20>

[GO-FMT]

Go Package Documentation, Package `fmt`. URL: <https://golang.org/pkg/fmt/>

[GOOGLE-BENCH]

Google Benchmark: A microbenchmark support library. URL: <https://github.com/google/benchmark>

[JAVA-PRINT]

Java™ Platform, Standard Edition 7 API Specification, Class `PrintStream`. URL: <https://docs.oracle.com/javase/7/docs/api/java/io/PrintStream.html>

[MSVC-UTF8]

Visual C++ Documentation, [/utf-8 \(Set Source and Executable character sets to UTF-8\)](https://docs.microsoft.com/en-us/cpp/build/reference/utf-8-set-source-and-executable-character-sets-to-utf-8). URL: <https://docs.microsoft.com/en-us/cpp/build/reference/utf-8-set-source-and-executable-character-sets-to-utf-8>

[N0147]

ISO/IEC IS 1989:2001 – Programming language COBOL, 14.8.10 `DISPLAY` statement. URL: https://web.archive.org/web/20020124065139/http://www.ncits.org/tc_home/j4htm/cobolv200112.zip

[N2162]

ISO/IEC 1539-1:2018 Information technology — Programming languages — Fortran.

[N2176]

ISO/IEC 9899:2017 Programming languages — C, 7.21.6.3. The `fprintf` function.

[P0645]

Victor Zverovich. [Text Formatting](https://wg21.link/p0645). URL: <https://wg21.link/p0645>

[P1885]

Corentin Jabot. [Naming Text Encodings to Demystify Them](https://wg21.link/p1885). URL: <https://wg21.link/p1885>

[PERL-PRINTF]

Perl 5 version 30.0 documentation, [Language reference, `printf`](https://perldoc.perl.org/functions/printf.html). URL: <https://perldoc.perl.org/functions/printf.html>

[PHP-PRINTF]

PHP Manual, Function Reference, printf. URL: <https://www.php.net/manual/en/function.printf.php>

[PY-FUNC]

The Python Standard Library, Built-in Functions. URL: <https://docs.python.org/3/library/functions.html>

[R-PRINT]

The R Core Team. R: A Language and Environment for Statistical Computing, Reference Index, printf. URL: <https://cran.r-project.org/doc/manuals/r-release/fullrefman.pdf#page=457>

[RUBY-PRINT]

Documentation for Ruby, print. URL: <https://docs.ruby-lang.org/en/2.7.0/ARGF.html#method-i-print>

[RUST-PRINT]

The Rust Standard Library, Macro std::print. URL: <https://doc.rust-lang.org/std/macro.print.html>

[SWIFT-PRINT]

Swift Standard Library, print. URL: <https://developer.apple.com/documentation/swift/1541053-print>

[WHATWG-CONSOLE]

WHATWG Standards, Console. URL: <https://console.spec.whatwg.org/>